

MVP System Architecture & Development Proposal

Arewa STEM Girls Artificial Intelligence Learning System

1. System Flow and Operational Logic (How Everything Connects)

The MVP system is will be constructed as a structured learning pipeline, where each component supports a continuous, low-friction journey from onboarding to knowledge acquisition and reinforcement.

At the user level, the learner interacts via an Android mobile application. The entry point into the system is authentication using a phone-based OTP mechanism. This approach reduces onboarding friction and aligns with the accessibility realities of the target demographic.

Once authenticated, the application communicates with the backend to retrieve the learner's profile. This includes:

- Age
- Learning level
- Language preference

These parameters directly influence content personalisation, ensuring that learners are not exposed to material outside their cognitive range.

The learner is then presented with a simplified dashboard, prioritising:

- "Continue Learning"
- Current course progress
- Recommended next lesson

When a lesson is accessed:

- Content is fetched from the backend content service
- The system caches the lesson locally on the device
- Subsequent access becomes independent of network stability

As the learner interacts with the system:

- Lesson completion is recorded locally
- Quiz attempts are stored locally
- Progress data is queued for synchronisation

Once connectivity is restored:

- A background sync engine updates the backend
- Data consistency is maintained without user disruption

The AI tutor interaction flow is tightly controlled:

- User submits a question
- Backend enriches the request with:
 - lesson context
 - learner level
- A constrained prompt is constructed
- The request is sent to the AI provider (GPT, Gemma, or Claude)
- The response is filtered and validated
- The final answer is delivered to the user

This establishes a controlled loop:

- Input: learner interaction
- Processing: backend logic + AI mediation
- Output: structured response
- Feedback: progress update

2. Development Stack

User Layer

Flutter (Android-first deployment):

- Efficient rendering on low-end devices
- Reduced fragmentation issues
- Strong offline support capabilities

Backend Layer

Python with FastAPI:

- High-performance API handling
- Native compatibility with AI workflows

Database Layer

PostgreSQL:

- Strong relational data modelling
- Scales effectively with user growth

Storage Layer

Firebase Storage

- Efficient delivery of media assets
- Simplified integration with mobile apps

Authentication

Firebase Authentication (OTP-based)

- Secure and user-friendly

Notifications

Firebase Cloud Messaging

- Enables engagement and reminders

AI Integration

Single LLM API provider via backend

Enables:

- cost control
- safety filtering
- request throttling

3. MVP Structure and Functions

1. Onboarding and Identity

- Phone-based registration (OTP)
- Profile setup:
 - age
 - level
 - language
 - state

2. Learning Module

- Structured course delivery
- Text-first lesson format
- Visual aids (images/illustrations)

3. Assessment Module

- Lesson-linked quizzes
- Immediate scoring and feedback
- Reinforcement of concepts

4. Progress Tracking

- Tracks:
 - lesson completion
 - quiz performance
- Displays:
 - progress indicators
 - achievement milestones

5. AI Tutor

- Text-based interaction only
- Context-aware responses
- Limited usage per user
- Curriculum-constrained outputs

Administrative Component

Admin Dashboard (Web-Based)

Manage:

- courses
- lessons
- quizzes

Monitor:

- user activity
- engagement metrics

Perform:

- basic moderation
- data export

4. MVP User Experience Flow and Design Logic

Onboarding Flow

- email address
- Language selection
- Profile setup
- Initial skill categorisation

Dashboard Design

Focus on:

- “Continue Learning”
- minimal distractions

Lesson Experience

- Clean, readable interface
- Linear progression

Assessment Experience

- Immediate feedback
- Clear explanations

5. Proposed Product Development Phases and Execution Strategy

Phase 0: Planning and Design (4 days)

- Stakeholder requirements finalization
- Design UI/UX
- Define system architecture
- Prepare content outline

Phase 1: Core System Development (2 Weeks)

- Authentication system
- Content delivery engine
- Lesson interface
- Quiz engine
- Progress tracking

Phase 2: AI Integration (1 Week)

- RAG AI tutor backend
- prompt constraints
- safety filters
- rate limits

Phase 3: Testing and Deployment (2 Weeks)

- Functional testing
- Performance optimisation

- Bug fixes
- Deployment

Total Estimated Timeline

- 6 weeks

Deferred for Future Phases

- Multi-role systems (parents, teachers)
- Advanced AI capabilities
- Community features
- Real-time collaboration tools

Deliverables

Technical Deliverables

- System architecture diagram
- Database schema documentation
- API documentation

Product Deliverables

- Source Code
- Android application (APK)
- Web-based admin dashboard

AI & System Documentation

- AI prompt and safety logic
- Rate limiting and usage controls

Note: This document represents a proposed system architecture. All components and design decisions are provisional and subject to modification, enhancement, or replacement as part of iterative development and optimisation processes.